Krylov Subspace Methods: Arnoldi, GMRES, and the Road to Implicit Layers

Lecture notes for MATH 616, Computational Linear Algebra
Dr. Xuping Xie, Old Dominion University

Fall 2026

Contents

1 Matrix-free problems and Krylov subspaces **2**

1.1 When you can multiply by A but not factor it 2

1.2 The Krylov subspace 3

1.3 The solution lies in a Krylov subspace 3

1.4 Stationary iterations live in Krylov subspaces 4

2 The Arnoldi iteration **5**

2.1 The monomial basis is useless 5

2.2 The algorithm 5

2.3 The Arnoldi relation 6

2.4 Breakdown 7

2.5 A worked example 7

2.6 Arnoldi, Hessenberg reduction, and eigenvalues 8

3 GMRES **9**

3.1 The minimal residual principle 9

3.2 Reduction to a small least squares problem 9

3.3 Updating the QR factorization with Givens rotations 10

3.4 The algorithm 10

3.5 The worked example, continued 11

3.6 Convergence 12

3.7 Cost, memory, restarts, and preconditioning 14

3.8 Symmetric matrices: MINRES and conjugate gradients 15

4 From linear systems to fixed points: toward deep equilibrium models **15**

4.1 Fixed-point problems 15

4.2 Forward solvers 16

4.3 What automatic differentiation provides 17

4.4 Implicit differentiation 18

4.5 Neumann series versus GMRES for the backward solve 19

4.6 Matrix-free GMRES in SciPy 20

4.7 Checklist: what the deep equilibrium lectures assume 21

5 Practice problems **22**

Solutions **23**

References **23**

1 Matrix-free problems and Krylov subspaces

1.1 When you can multiply by A but not factor it

Every method we have studied so far for $Ax = b$ needs the entries of A: Gaussian elimination and Cholesky factor it, and Jacobi, Gauss–Seidel and SOR split it into $D - L - U$. There are important problems in which A is available only as a *black box* that maps a vector v to the product Av:

- A is sparse and enormous, so that a factorization would fill in and run out of memory, but one product Av costs only as much as the number of nonzeros.
- A is the Jacobian of a nonlinear map f at some point. Automatic differentiation computes Jacobian–vector products Jv and vector–Jacobian products $J^{\mathsf{T}}u$ at the cost of about one evaluation of f, without ever forming J. This is the case in deep equilibrium models (Section 4).
- A is a product or inverse of other operators, for example $A = B^{-1}C$, where $B^{-1}Cv$ is computed by one solve with B.

Methods that use A only through products Av are called *matrix-free*. The stationary iterations of the previous unit (Jacobi, Gauss–Seidel, SOR) are not quite matrix-free, since they need the diagonal or triangular parts of A. The simplest matrix-free iteration is *Richardson's method*

$$x_{k+1} = x_k + \omega\,(b - Ax_k), \tag{1.1}$$

which is the fixed-point iteration for $x = x + \omega(b - Ax)$. Its iteration matrix is $M = I - \omega A$, and by the convergence theorem of the previous unit it converges for every starting vector if and only if $\rho(I - \omega A) < 1$. Krylov methods start from the observation that Richardson's method, and every other method built from products with A, can only ever produce vectors in a very specific subspace.

1.2 The Krylov subspace

Definition 1.1. For $A \in \mathbb{C}^{n\times n}$, $b \in \mathbb{C}^n$ and $k \geq 1$, the k-th *Krylov subspace* is

$$\mathcal{K}_k(A, b) = \operatorname{span}\{b,\, Ab,\, A^2b,\, \ldots,\, A^{k-1}b\}.$$

The subspaces are nested, $\mathcal{K}_1 \subseteq \mathcal{K}_2 \subseteq \cdots$, and $\dim \mathcal{K}_k \leq k$. Their defining property is that they consist of polynomials in A applied to b.

Lemma 1.2. *$x \in \mathcal{K}_k(A, b)$ if and only if $x = q(A)\,b$ for some polynomial q of degree at most $k-1$.*

Proof. $x = \sum_{j=0}^{k-1} c_j A^j b$ is the same statement as $x = q(A)b$ with $q(t) = \sum_{j=0}^{k-1} c_j t^j$. □

1.3 The solution lies in a Krylov subspace

Why should we look for the solution of $Ax = b$ in $\mathcal{K}_k(A, b)$ at all? Because, for k large enough, it is there.

Theorem 1.3. *Let A be nonsingular. There is an integer $m \leq n$, called the* grade *of b with respect to A, such that*

$$\mathcal{K}_1 \subsetneq \mathcal{K}_2 \subsetneq \cdots \subsetneq \mathcal{K}_m = \mathcal{K}_{m+1} = \mathcal{K}_{m+2} = \cdots,$$

$\dim \mathcal{K}_k = k$ for $k \leq m$, $\mathcal{K}_m$ is invariant under A, and $A^{-1}b \in \mathcal{K}_m(A, b)$.

Proof. Since $\mathcal{K}_k \subseteq \mathcal{K}_{k+1}$ and the dimensions are bounded by n, the dimension must stop growing at some first index m, where $A^m b \in \mathcal{K}_m$. Then $A\mathcal{K}_m \subseteq \mathcal{K}_m$: a basis vector $A^j b$ with $j < m-1$ is mapped to $A^{j+1}b \in \mathcal{K}_m$, and $A^{m-1}b$ is mapped to $A^m b \in \mathcal{K}_m$. Hence $\mathcal{K}_{m+1} = \mathcal{K}_m + A\mathcal{K}_m = \mathcal{K}_m$, and by induction all later subspaces coincide. Since $A^m b \in \mathcal{K}_m$ there are coefficients with

$$A^m b = c_0 b + c_1 Ab + \cdots + c_{m-1} A^{m-1} b.$$

Here $c_0 \neq 0$: otherwise $A(A^{m-1}b - c_1 b - \cdots - c_{m-1}A^{m-2}b) = 0$, and since A is nonsingular this would make $A^{m-1}b \in \mathcal{K}_{m-1}$, contradicting the minimality of m. Multiplying by A^{-1} and solving for $A^{-1}b$,

$$A^{-1}b = \frac{1}{c_0}\left(A^{m-1}b - c_1 b - c_2 Ab - \cdots - c_{m-1}A^{m-2}b\right) \in \mathcal{K}_m.$$

□

The polynomial $t^m - c_{m-1}t^{m-1} - \cdots - c_0$ in the proof is the *minimal polynomial of b with respect to A*. By the Cayley–Hamilton theorem the characteristic polynomial of A annihilates b, so $m \leq n$ always; and if A has only d distinct eigenvalues and is diagonalizable, then $m \leq d$. This is the first hint of a principle that governs all Krylov methods: *few distinct, or tightly clustered, eigenvalues mean fast convergence.*

Example 1.4 (Grade smaller than n)**.** Let $A = \operatorname{diag}(1,1,2)$ and $b = (1,1,1)^{\mathsf{T}}$. Then $Ab = (1,1,2)^{\mathsf{T}}$ and $A^2b = (1,1,4)^{\mathsf{T}} = 3Ab - 2b$, so $A^2b \in \mathcal{K}_2$ and the grade is $m = 2 < 3$. In the notation of the proof, $c_0 = -2$ and $c_1 = 3$, and

$$A^{-1}b = \frac{1}{c_0}\big(Ab - c_1 b\big) = -\tfrac{1}{2}\big((1,1,2)^{\mathsf{T}} - 3(1,1,1)^{\mathsf{T}}\big) = (1,1,\tfrac{1}{2})^{\mathsf{T}},$$

which is correct. The minimal polynomial of b is $t^2 - 3t + 2 = (t-1)(t-2)$: it has one root per *distinct* eigenvalue, and the repeated eigenvalue 1 costs nothing. A Krylov solver will find $A^{-1}b$ in two steps.

1.4 Stationary iterations live in Krylov subspaces

Theorem 1.5. *The Richardson iterates* (1.1) *started from* $x_0 = 0$ *satisfy* $x_k \in \mathcal{K}_k(A,b)$ *for every* $k \geq 1$.

Proof. By induction. $x_1 = \omega b \in \mathcal{K}_1$. If $x_k = q(A)b$ with $\deg q \leq k-1$, then

$$x_{k+1} = x_k + \omega b - \omega A x_k = \big(q(A) + \omega I - \omega A q(A)\big)b,$$

and the polynomial $q(t) + \omega - \omega t\, q(t)$ has degree at most k. □

Writing out the iterates gives the *Neumann series*: with $x_0 = 0$,

$$x_k = \omega \sum_{j=0}^{k-1} (I - \omega A)^j b \;\longrightarrow\; \omega\big(I - (I - \omega A)\big)^{-1} b = A^{-1}b \qquad \text{if } \rho(I - \omega A) < 1. \tag{1.2}$$

The Jacobi iteration $x_{k+1} = x_k + D^{-1}(b - Ax_k)$ is Richardson's method with $\omega = 1$ applied to the scaled system $D^{-1}Ax = D^{-1}b$, so its iterates lie in $\mathcal{K}_k(D^{-1}A, D^{-1}b)$; Gauss–Seidel and SOR are similar with $(D-L)^{-1}$ in place of D^{-1}.

Example 1.6 (A Neumann series by hand)**.** Let $A = I - J$ with $J = \begin{pmatrix} 0 & \frac{1}{2} \\ \frac{1}{2} & 0 \end{pmatrix}$ and $b = (1,0)^{\mathsf{T}}$. Richardson's method with $\omega = 1$ is $x_{k+1} = x_k + (b - Ax_k) = Jx_k + b$, and from $x_0 = 0$:

$$x_1 = (1,0)^{\mathsf{T}}, \quad x_2 = (1,\tfrac{1}{2})^{\mathsf{T}}, \quad x_3 = (\tfrac{5}{4},\tfrac{1}{2})^{\mathsf{T}}, \quad x_4 = (\tfrac{5}{4},\tfrac{5}{8})^{\mathsf{T}}, \quad x_5 = (\tfrac{21}{16},\tfrac{5}{8})^{\mathsf{T}}, \ \ldots$$

converging to $A^{-1}b = \frac{4}{3}(1,\frac{1}{2})^{\mathsf{T}} = (\frac{4}{3},\frac{2}{3})^{\mathsf{T}}$. The residual $b - Ax_k = J^k b$ has norm exactly 2^{-k}, because $\rho(J) = \frac{1}{2}$ and $J^k b$ alternates between $2^{-k}e_1$ and $2^{-k}e_2$. The eigenvalues of A are $\frac{1}{2}$ and $\frac{3}{2}$, only two distinct values, so the grade of b is 2 and a Krylov method will solve the system exactly in two products with A, where Richardson's method is still at residual $\frac{1}{4}$. We return to this example in Section 3.

So a stationary iteration picks one particular vector from $\mathcal{K}_k$ at step k, namely the one dictated by the fixed recursion. A *Krylov subspace method* instead picks the *best* vector in $\mathcal{K}_k$ according to some criterion. Whatever the criterion, two things are needed: a good basis of $\mathcal{K}_k$ (Section 2) and a cheap way to find the best vector in it (Section 3).

2 The Arnoldi iteration

2.1 The monomial basis is useless

The vectors $b, Ab, A^2b, \ldots$ span $\mathcal{K}_k$, but they are a terrible basis to compute with. The power method told us why: A^jb converges, after scaling, to the dominant eigenvector of A. So for moderate j the vectors A^jb and $A^{j+1}b$ point in nearly the same direction, and the basis is numerically as bad as the columns of a Vandermonde matrix. The cure is the same as in least squares: orthonormalize. Orthonormalizing the Krylov sequence by Gram–Schmidt, one vector at a time, is the Arnoldi iteration.

Example 2.1 (How bad the monomial basis is)**.** Let A be the 100×100 tridiagonal matrix with 2 on the diagonal and -1 on both off-diagonals (the one-dimensional Laplacian from the direct methods unit), and $b = (1, \ldots, 1)^{\mathsf{T}}$. Normalize each of $b, Ab, \ldots, A^{k-1}b$ to unit length and put them in the columns of a matrix K_k. Its 2-norm condition number grows as follows.

k	$\kappa_2(K_k)$
3	4.3
5	9.7×10^1
8	1.4×10^4
10	4.2×10^5
15	2.3×10^9
20	1.3×10^{13}

By $k = 20$ the basis is numerically rank deficient in double precision, even though the subspace it spans is perfectly fine. The Arnoldi basis of the same subspace has condition number 1.

2.2 The algorithm

Start with $q_1 = b/\|b\|_2$. Having built orthonormal $q_1, \ldots, q_k$ spanning $\mathcal{K}_k$, the next vector of the Krylov sequence that we need is not A^kb but simply Aq_k, because $Aq_k \in \mathcal{K}_{k+1}$ and it is not in $\mathcal{K}_k$ unless the space has become invariant. Orthogonalize Aq_k against $q_1, \ldots, q_k$ and normalize:

$$\begin{aligned} v &= Aq_k, \\ h_{jk} &= q_j^{\mathsf{T}} v, \qquad v \leftarrow v - h_{jk} q_j \qquad (j = 1, \ldots, k), \\ h_{k+1,k} &= \|v\|_2, \qquad q_{k+1} = v / h_{k+1,k}. \end{aligned}$$

The inner loop is *modified* Gram–Schmidt: each h_{jk} is computed from the partially orthogonalized v, which is more stable than computing all inner products from the original Aq_k.

```
import numpy as np

def arnoldi(matvec, b, k):
    """k steps of the Arnoldi iteration for the Krylov space K_k(A, b).

    matvec(v) must return A @ v.  Returns Q (n x (k+1)) with orthonormal
    columns and H ((k+1) x k, upper Hessenberg) with  A Q[:, :k] = Q H.
```

```
    Stops early (breakdown) if the Krylov space becomes invariant.
    """
    n = b.size
    Q = np.zeros((n, k + 1))
    H = np.zeros((k + 1, k))
    Q[:, 0] = b / np.linalg.norm(b)
    for j in range(k):
        v = matvec(Q[:, j])
        for i in range(j + 1):                  # modified Gram-Schmidt
            H[i, j] = Q[:, i] @ v
            v = v - H[i, j] * Q[:, i]
        H[j + 1, j] = np.linalg.norm(v)
        if H[j + 1, j] < 1e-12 * np.linalg.norm(b):   # breakdown
            return Q[:, :j + 1], H[:j + 1, :j + 1]
        Q[:, j + 1] = v / H[j + 1, j]
    return Q, H
```

Step k costs one product with A plus $O(kn)$ flops for the orthogonalization, and the algorithm stores all $k+1$ basis vectors.

2.3 The Arnoldi relation

Collect the basis vectors in $Q_k = (q_1 \ \cdots \ q_k) \in \mathbb{R}^{n \times k}$ and the coefficients in the $(k+1) \times k$ matrix

$$\tilde{H}_k = \begin{pmatrix} h_{11} & h_{12} & \cdots & h_{1k} \\ h_{21} & h_{22} & \cdots & h_{2k} \\ & h_{32} & \ddots & \vdots \\ & & \ddots & h_{kk} \\ & & & h_{k+1,k} \end{pmatrix},$$

which is upper Hessenberg with one extra row. Let H_k be its top $k \times k$ block.

Theorem 2.2 (Arnoldi relation)**.** *If no breakdown occurs in the first k steps, then $q_1, \dots, q_{k+1}$ are orthonormal,* $\operatorname{span}\{q_1, \dots, q_j\} = \mathcal{K}_j(A, b)$ *for $j \le k+1$, and*

$$AQ_k = Q_{k+1}\tilde{H}_k = Q_k H_k + h_{k+1,k}\, q_{k+1} e_k^{\mathsf{T}}, \qquad Q_k^{\mathsf{T}} A Q_k = H_k. \tag{2.1}$$

Proof. Orthonormality is what Gram–Schmidt guarantees, and the span statement follows by induction because q_{j+1} is a nonzero multiple of $Aq_j - \sum_{i \le j} h_{ij} q_i$ with $Aq_j \in \mathcal{K}_{j+1} \setminus \mathcal{K}_j$. Rearranging the k-th step of the algorithm gives

$$Aq_k = \sum_{j=1}^{k} h_{jk} q_j + h_{k+1,k} q_{k+1},$$

which is column k of $AQ_k = Q_{k+1}\tilde{H}_k$. Splitting off the last term of the sum gives the second form. Multiplying by Q_k^{T} and using $Q_k^{\mathsf{T}} Q_k = I$, $Q_k^{\mathsf{T}} q_{k+1} = 0$ gives $Q_k^{\mathsf{T}} A Q_k = H_k$. □

The second equality in (2.1) says that H_k is the *orthogonal projection of A onto $\mathcal{K}_k$*, expressed in the basis Q_k. Everything a Krylov method knows about A after k steps is contained in the small Hessenberg matrix $\tilde{H}_k$.

2.4 Breakdown

Lemma 2.3. *$h_{k+1,k} = 0$ if and only if $\mathcal{K}_k(A, b)$ is invariant under A, that is, k is the grade of b. In that case $AQ_k = Q_k H_k$, and if A is nonsingular then $A^{-1}b \in \mathcal{K}_k$.*

Proof. $h_{k+1,k} = 0$ means $Aq_k \in \operatorname{span}\{q_1, \ldots, q_k\} = \mathcal{K}_k$. Since $Aq_j \in \mathcal{K}_{j+1} \subseteq \mathcal{K}_k$ for $j < k$ anyway, this is the same as $A\mathcal{K}_k \subseteq \mathcal{K}_k$. The rest is Theorem 1.3. □

For a linear solver, breakdown is good news (the exact solution is in reach), which is why it is called a *lucky breakdown*.

2.5 A worked example

Take

$$A = \begin{pmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{pmatrix}, \qquad b = e_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}.$$

Step 1. $q_1 = e_1$. $Aq_1 = (2, -1, 0)^{\mathsf{T}}$, $h_{11} = q_1^{\mathsf{T}} Aq_1 = 2$, $v = (2, -1, 0)^{\mathsf{T}} - 2e_1 = (0, -1, 0)^{\mathsf{T}}$, $h_{21} = 1$, $q_2 = (0, -1, 0)^{\mathsf{T}}$.

Step 2. $Aq_2 = (1, -2, 1)^{\mathsf{T}}$. $h_{12} = q_1^{\mathsf{T}} Aq_2 = 1$, $v = (1, -2, 1)^{\mathsf{T}} - 1 \cdot q_1 = (0, -2, 1)^{\mathsf{T}}$; $h_{22} = q_2^{\mathsf{T}} v = 2$, $v \leftarrow (0, -2, 1)^{\mathsf{T}} - 2q_2 = (0, 0, 1)^{\mathsf{T}}$; $h_{32} = 1$, $q_3 = (0, 0, 1)^{\mathsf{T}}$.

Step 3. $Aq_3 = (0, -1, 2)^{\mathsf{T}}$. $h_{13} = 0$, $h_{23} = q_2^{\mathsf{T}} Aq_3 = 1$, $v = (0, -1, 2)^{\mathsf{T}} - q_2 = (0, 0, 2)^{\mathsf{T}}$, $h_{33} = 2$, $v \leftarrow (0, 0, 2)^{\mathsf{T}} - 2q_3 = 0$. So $h_{43} = 0$: breakdown, as it must be since $k = n = 3$.

The result is

$$Q_3 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \qquad H_3 = \begin{pmatrix} 2 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 2 \end{pmatrix},$$

and one checks $Q_3^{\mathsf{T}} AQ_3 = H_3$. Note that H_3 is tridiagonal: for symmetric A the Hessenberg matrix $H_k = Q_k^{\mathsf{T}} AQ_k$ is symmetric, hence tridiagonal.

Example 2.4 (A nonsymmetric matrix)**.** Take

$$A = \begin{pmatrix} 2 & 1 & 0 \\ 0 & 3 & 1 \\ 1 & 0 & 4 \end{pmatrix}, \qquad b = e_1.$$

Step 1. $q_1 = e_1$, $Aq_1 = (2, 0, 1)^{\mathsf{T}}$, $h_{11} = 2$, $v = (0, 0, 1)^{\mathsf{T}}$, $h_{21} = 1$, $q_2 = e_3$.

Step 2. $Aq_2 = (0, 1, 4)^{\mathsf{T}}$, $h_{12} = q_1^{\mathsf{T}} Aq_2 = 0$, $h_{22} = q_2^{\mathsf{T}} Aq_2 = 4$, $v = (0, 1, 4)^{\mathsf{T}} - 4e_3 = (0, 1, 0)^{\mathsf{T}}$, $h_{32} = 1$, $q_3 = e_2$.

Step 3. $Aq_3 = (1, 3, 0)^{\mathsf{T}}$, $h_{13} = 1$, $h_{23} = 0$, $h_{33} = 3$, $v = (1, 3, 0)^{\mathsf{T}} - e_1 - 3e_2 = 0$, breakdown.

So

$$Q_3 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}, \qquad H_3 = \begin{pmatrix} 2 & 0 & 1 \\ 1 & 4 & 0 \\ 0 & 1 & 3 \end{pmatrix},$$

and $H_3 = Q_3^{\mathsf{T}} A Q_3$ is a permutation of A. This time H_3 is upper Hessenberg but not tridiagonal: the entry $h_{13} = 1$ above the superdiagonal is the signature of a nonsymmetric matrix. In general every q_j with $j \le k$ contributes to the orthogonalization of Aq_k, which is why Arnoldi must store the whole basis while Lanczos, for symmetric A, needs only the last two vectors.

2.6 Arnoldi, Hessenberg reduction, and eigenvalues

Remark 2.5. In the QR algorithm unit we reduced A to Hessenberg form $Q^{\mathsf{T}} A Q = H$ with Householder reflectors. The Arnoldi iteration computes the first k columns of exactly that factorization, for the particular choice $q_1 = b/\|b\|_2$ (Hessenberg reduction with a prescribed first column is unique up to signs). The difference is the purpose: Householder reduction processes the whole matrix in $O(n^3)$ flops, while Arnoldi stops after $k \ll n$ steps and touches A only through k products.

Remark 2.6 (Ritz values). The eigenvalues of the small matrix $H_k = Q_k^{\mathsf{T}} A Q_k$ are called *Ritz values*. They are the Rayleigh quotient idea applied to a subspace, and they are often excellent approximations to the extreme eigenvalues of A long before k approaches n. This is the basis of the Arnoldi eigenvalue solver (ARPACK, called by `scipy.sparse.linalg.eigs`); for symmetric matrices the three-term version is the Lanczos iteration (`eigsh`). We will not pursue eigenvalues further here; see Trefethen and Bau, Lectures 33–36.

Example 2.7 (Ritz values of the Laplacian)**.** For the 100×100 Laplacian matrix above, the eigenvalues are $2 - 2\cos(j\pi/101)$, so $\lambda_{\min} \approx 0.000967$ and $\lambda_{\max} \approx 3.99903$. Running Arnoldi from a random b and computing the smallest and largest eigenvalues of H_k gives:

k	Smallest Ritz value	Largest Ritz value
5	0.135747	3.876517
10	0.047889	3.971446
20	0.002714	3.990957
40	0.000986	3.998358

With 40 products, less than half the dimension, both extreme eigenvalues are captured to three digits, and the largest Ritz value is already between the two largest true eigenvalues (3.99613 and 3.99903). Ritz values approach the spectrum from the inside: the largest Ritz value never exceeds $\lambda_{\max}$ and the smallest never drops below $\lambda_{\min}$, because $H_k = Q_k^{\mathsf{T}} A Q_k$ is a compression of A (Courant–Fischer, from the midterm review).

3 GMRES

3.1 The minimal residual principle

GMRES (generalized minimal residual, Saad and Schultz 1986) chooses the vector in the Krylov subspace with the smallest residual:

$$x_k = \underset{x \in \mathcal{K}_k(A,b)}{\arg\min} \ \|b - Ax\|_2. \tag{3.1}$$

(We take $x_0 = 0$ throughout; for a general starting vector replace b by $r_0 = b - Ax_0$ and add x_0 to the result.) Two properties are immediate:

- *Monotonicity.* Since $\mathcal{K}_k \subseteq \mathcal{K}_{k+1}$, the residual norms $\|r_k\|_2$ are non-increasing.
- *Finite termination.* By Theorem 1.3, $A^{-1}b \in \mathcal{K}_m$ with $m \le n$, so $r_m = 0$: in exact arithmetic GMRES is a direct method. In practice we hope to stop at $k \ll m$.

By Lemma 1.2, $x = q(A)b$ with $\deg q \le k-1$ and the residual is $r = b - Aq(A)b = p(A)b$ with $p(t) = 1 - tq(t)$. So GMRES solves a *polynomial approximation problem*:

$$\|r_k\|_2 = \min\bigl\{\|p(A)b\|_2 : \ \deg p \le k, \ p(0) = 1\bigr\}. \tag{3.2}$$

This is the form in which we will analyze convergence.

3.2 Reduction to a small least squares problem

The minimization (3.1) is over an n-dimensional vector space, but the Arnoldi relation collapses it to a problem of size $(k+1) \times k$.

Theorem 3.1. *Let $\beta = \|b\|_2$, and let Q_{k+1}, $\tilde{H}_k$ be produced by k steps of Arnoldi without breakdown. Then the GMRES iterate is $x_k = Q_k y_k$, where*

$$y_k = \underset{y \in \mathbb{R}^k}{\arg\min} \ \|\beta e_1 - \tilde{H}_k y\|_2, \tag{3.3}$$

and $\|b - Ax_k\|_2 = \|\beta e_1 - \tilde{H}_k y_k\|_2$.

Proof. Every $x \in \mathcal{K}_k$ is $x = Q_k y$ for a unique $y \in \mathbb{R}^k$. Using $b = \beta q_1 = Q_{k+1}(\beta e_1)$ and the Arnoldi relation $AQ_k = Q_{k+1}\tilde{H}_k$,

$$b - Ax = Q_{k+1}\beta e_1 - Q_{k+1}\tilde{H}_k y = Q_{k+1}(\beta e_1 - \tilde{H}_k y).$$

Since Q_{k+1} has orthonormal columns, $\|Q_{k+1}z\|_2 = \|z\|_2$ for every $z \in \mathbb{R}^{k+1}$, so minimizing $\|b - Ax\|_2$ over $x \in \mathcal{K}_k$ is the same as minimizing $\|\beta e_1 - \tilde{H}_k y\|_2$ over $y \in \mathbb{R}^k$. □

The matrix $\tilde{H}_k$ has full column rank as long as $h_{k+1,k} \neq 0$, so the least squares problem has a unique solution, which can be found by QR factorization of $\tilde{H}_k$.

3.3 Updating the QR factorization with Givens rotations

Because $\tilde{H}_k$ is Hessenberg, its QR factorization needs only k Givens rotations, one per subdiagonal entry, and the factorization of $\tilde{H}_{k+1}$ is obtained from that of $\tilde{H}_k$ by one more rotation. This is the same tool we used for a QR step on a tridiagonal matrix.

Suppose rotations $G_1, \dots, G_{k-1}$ (each acting in the plane of rows $j, j+1$) have been found with

$$G_{k-1} \cdots G_1 \tilde{H}_{k-1} = \begin{pmatrix} R_{k-1} \\ 0 \end{pmatrix}, \qquad G_{k-1} \cdots G_1 (\beta e_1) = g = (g_1, \dots, g_k)^{\mathsf{T}}.$$

When Arnoldi produces column k of $\tilde{H}_k$, apply the old rotations to it, which fills the column down to row $k+1$ with entries $\tilde{h}_{1k}, \dots, \tilde{h}_{kk}, h_{k+1,k}$. One new rotation with

$$c_k = \frac{\tilde{h}_{kk}}{\sqrt{\tilde{h}_{kk}^2 + h_{k+1,k}^2}}, \qquad s_k = \frac{h_{k+1,k}}{\sqrt{\tilde{h}_{kk}^2 + h_{k+1,k}^2}}$$

zeroes $h_{k+1,k}$ and, applied to g, replaces its last entry g_k by $c_k g_k$ and appends $g_{k+1} = -s_k g_k$. Since the rotations are orthogonal,

$$\|\beta e_1 - \tilde{H}_k y\|_2 = \left\| \begin{pmatrix} g_{1:k} \\ g_{k+1} \end{pmatrix} - \begin{pmatrix} R_k \\ 0 \end{pmatrix} y \right\|_2,$$

which is minimized by solving the triangular system $R_k y = g_{1:k}$, leaving the residual norm $|g_{k+1}|$.

Proposition 3.2. *With this update, the GMRES residual norm after k steps is $\|r_k\|_2 = |g_{k+1}| = \beta\,|s_1 s_2 \cdots s_k|$, and it is available* without forming x_k. *The iterate $x_k = Q_k y_k$ needs to be computed only once, when the residual is small enough.*

3.4 The algorithm

```
import numpy as np

def gmres(matvec, b, tol=1e-8, maxit=50):
    """GMRES for A x = b with x0 = 0, using only matvec(v) = A @ v.

    Arnoldi basis is built one vector per step; the small least squares
    problem is solved with numpy's lstsq for clarity (production codes
    update a QR factorization with Givens rotations instead).
    Returns x and the list of residual norms ||b - A x_k||_2.
    """
    n = b.size
    beta = np.linalg.norm(b)
    Q = np.zeros((n, maxit + 1))
    H = np.zeros((maxit + 1, maxit))
    Q[:, 0] = b / beta
    resids = [beta]
    for k in range(maxit):
```

```
        v = matvec(Q[:, k])
        for i in range(k + 1):
            H[i, k] = Q[:, i] @ v
            v = v - H[i, k] * Q[:, i]
        H[k + 1, k] = np.linalg.norm(v)
        if H[k + 1, k] > 1e-14 * beta:
            Q[:, k + 1] = v / H[k + 1, k]
        e1 = np.zeros(k + 2); e1[0] = beta
        y, *_ = np.linalg.lstsq(H[:k + 2, :k + 1], e1, rcond=None)
        resids.append(np.linalg.norm(e1 - H[:k + 2, :k + 1] @ y))
        if resids[-1] <= tol * beta or H[k + 1, k] <= 1e-14 * beta:
            break
    x = Q[:, :k + 1] @ y
    return x, resids
```

In SciPy the same algorithm, with Givens updates and restarts, is `scipy.sparse.linalg.gmres`, and a matrix-free operator is wrapped as a `LinearOperator` (Section 4.6).

3.5 The worked example, continued

Continue the example of Section 2.5, where $\beta = 1$.

$k = 1$. $\tilde{H}_1 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$. Minimize $\|(1,0)^{\mathsf{T}} - y(2,1)^{\mathsf{T}}\|_2$: the normal equation $5y = 2$ gives $y_1 = 2/5$, residual $(1 - 4/5,\, -2/5)^{\mathsf{T}}$ with norm $\sqrt{1/25 + 4/25} = 1/\sqrt{5} \approx 0.447$. So $x_1 = \frac{2}{5} q_1 = (0.4, 0, 0)^{\mathsf{T}}$.

$k = 2$. $\tilde{H}_2 = \begin{pmatrix} 2 & 1 \\ 1 & 2 \\ 0 & 1 \end{pmatrix}$. The normal equations $\tilde{H}_2^{\mathsf{T}} \tilde{H}_2 y = \tilde{H}_2^{\mathsf{T}} e_1$ read

$$\begin{pmatrix} 5 & 4 \\ 4 & 6 \end{pmatrix} y = \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \qquad y_2 = \frac{1}{14} \begin{pmatrix} 6 & -4 \\ -4 & 5 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 4/7 \\ -3/14 \end{pmatrix}.$$

Then $\tilde{H}_2 y_2 = (13/14,\, 2/14,\, -3/14)^{\mathsf{T}}$, the residual is $(1/14,\, -2/14,\, 3/14)^{\mathsf{T}}$ with norm $\sqrt{14}/14 = 1/\sqrt{14} \approx 0.267$, and

$$x_2 = Q_2 y_2 = \tfrac{4}{7} q_1 - \tfrac{3}{14} q_2 = \left(\tfrac{4}{7},\ \tfrac{3}{14},\ 0\right)^{\mathsf{T}} \approx (0.571,\ 0.214,\ 0)^{\mathsf{T}}.$$

$k = 3$. Breakdown occurred, so $\tilde{H}_3 = H_3$ is square and the least squares problem is the linear system $H_3 y = e_1$, with solution $y_3 = (3/4,\, -1/2,\, 1/4)^{\mathsf{T}}$ and residual zero. Hence $x_3 = Q_3 y_3 = (3/4,\, 1/2,\, 1/4)^{\mathsf{T}}$, which is indeed $A^{-1} e_1$.

For comparison, run Richardson's method (1.1) with $\omega = 1/2$, which for this matrix is exactly the Jacobi iteration because $D = 2I$: $x_1 = (0.5, 0, 0)^{\mathsf{T}}$, $x_2 = (0.5, 0.25, 0)^{\mathsf{T}}$, $x_3 = (0.625, 0.25, 0.125)^{\mathsf{T}}$. Its iterates lie in the same Krylov subspaces, and the residuals compare as follows.

k	GMRES $\|r_k\|_2$	Jacobi (Richardson, $\omega = \frac{1}{2}$) $\|r_k\|_2$
1	0.447	0.500
2	0.267	0.354
3	0	0.250

GMRES wins at every step, and it must: it picks the best vector in a subspace that contains the Jacobi iterate.

Example 3.3 (The same residuals from Givens rotations)**.** Redo the example with the rotation update, to see the residual norms appear without any least squares solve. With $\beta = 1$ and $g = (1, 0)^{\mathsf{T}}$:

$k = 1$. Column 1 of $\tilde{H}_1$ is $(2, 1)^{\mathsf{T}}$, so $c_1 = 2/\sqrt{5}$, $s_1 = 1/\sqrt{5}$, and $R_1 = (\sqrt{5})$. Rotating g gives $g = (2/\sqrt{5},\ -1/\sqrt{5})^{\mathsf{T}}$, so $\|r_1\|_2 = |g_2| = 1/\sqrt{5} \approx 0.447$. (Check: $y_1 = g_1/R_1 = 2/5$, as before.)

$k = 2$. The new column is $(1, 2, 1)^{\mathsf{T}}$. Applying the first rotation to rows $1, 2$ gives $(c_1 \cdot 1 + s_1 \cdot 2,\ -s_1 \cdot 1 + c_1 \cdot 2,\ 1)^{\mathsf{T}} = (4/\sqrt{5},\ 3/\sqrt{5},\ 1)^{\mathsf{T}}$. The new rotation zeroes the last entry: with $\sqrt{9/5 + 1} = \sqrt{14/5}$, $c_2 = 3/\sqrt{14}$ and $s_2 = \sqrt{5}/\sqrt{14}$. Applying it to g replaces $g_2 = -1/\sqrt{5}$ by $c_2 g_2 = -3/\sqrt{70}$ and appends $g_3 = -s_2 g_2 = 1/\sqrt{14}$. So $\|r_2\|_2 = 1/\sqrt{14} \approx 0.267$, as computed before, and $\beta|s_1 s_2| = \frac{1}{\sqrt{5}} \cdot \frac{\sqrt{5}}{\sqrt{14}} = \frac{1}{\sqrt{14}}$ confirms the product formula.

Example 3.4 (GMRES on the nonsymmetric example)**.** For the matrix of Example 2.4 and $b = e_1$, with $\tilde{H}_2 = \begin{pmatrix} 2 & 0 \\ 1 & 4 \\ 0 & 1 \end{pmatrix}$, the normal equations are

$$\begin{pmatrix} 5 & 4 \\ 4 & 17 \end{pmatrix} y = \begin{pmatrix} 2 \\ 0 \end{pmatrix}, \qquad y_2 = \tfrac{1}{69} \begin{pmatrix} 34 \\ -8 \end{pmatrix} \approx \begin{pmatrix} 0.4928 \\ -0.1159 \end{pmatrix},$$

so $x_2 = \frac{34}{69} q_1 - \frac{8}{69} q_2 = (0.4928,\ 0,\ -0.1159)^{\mathsf{T}}$ and $\|r_2\|_2 = 1/\sqrt{69} \approx 0.120$. The residual norms are $1,\ 0.447,\ 0.120,\ 0$, and $x_3 = A^{-1} e_1 = (0.48,\ 0.04,\ -0.12)^{\mathsf{T}}$. Note that x_2 has a zero second component: $\mathcal{K}_2 = \operatorname{span}\{e_1, e_3\}$ does not contain e_2 at all, and the exact solution's second component is small (0.04), which is why two steps already give a good answer.

Example 3.5 (Example 1.6, finished)**.** For $A = I - J$ with $J = \begin{pmatrix} 0 & \frac{1}{2} \\ \frac{1}{2} & 0 \end{pmatrix}$ and $b = e_1$: $Ab = (1, -\frac{1}{2})^{\mathsf{T}}$, so the GMRES step 1 minimizes $\|e_1 - y(1, -\frac{1}{2})^{\mathsf{T}}\|_2$, giving $y = \frac{4}{5}$ and $\|r_1\|_2 = 1/\sqrt{5} \approx 0.447$, versus 0.5 for the Neumann series. At step 2 GMRES is exact, versus 0.25 for the Neumann series, because A has two distinct eigenvalues $(\frac{1}{2}, \frac{3}{2})$.

3.6 Convergence

Theorem 3.6. *Let $A = V \Lambda V^{-1}$ be diagonalizable with eigenvalues $\lambda_1, \ldots, \lambda_n$. Then the GMRES residuals satisfy*

$$\frac{\|r_k\|_2}{\|b\|_2} \ \le\ \kappa_2(V) \min_{\substack{\deg p \le k \\ p(0)=1}} \max_{1 \le i \le n} |p(\lambda_i)|.$$

Proof. By (3.2), $\|r_k\|_2 \le \|p(A) b\|_2 \le \|p(A)\|_2 \|b\|_2$ for every admissible p. Since $p(A) = V p(\Lambda) V^{-1}$ and $p(\Lambda) = \operatorname{diag}(p(\lambda_i))$,

$$\|p(A)\|_2 \le \|V\|_2 \, \|p(\Lambda)\|_2 \, \|V^{-1}\|_2 = \kappa_2(V) \max_i |p(\lambda_i)|.$$

Take the minimum over p. □

The theorem turns a question about matrices into a question about polynomials: *how small can a polynomial with $p(0) = 1$ be on the spectrum of A?* Three consequences:

Corollary 3.7 (Few distinct eigenvalues)**.** *If A is diagonalizable with d distinct eigenvalues, GMRES converges in at most d steps.*

Proof. $p(t) = \prod_{j=1}^{d}(1 - t/\mu_j)$, with μ_j the distinct eigenvalues, has degree d, $p(0) = 1$, and vanishes on the spectrum. □

Corollary 3.8 (Eigenvalues in a disk)**.** *If every eigenvalue lies in the disk $|\lambda - c| \le \rho$ with $\rho < |c|$, then*

$$\frac{\|r_k\|_2}{\|b\|_2} \le \kappa_2(V)\left(\frac{\rho}{|c|}\right)^k.$$

Proof. Use $p(t) = (1 - t/c)^k$: $|p(\lambda_i)| = |c - \lambda_i|^k/|c|^k \le (\rho/|c|)^k$. □

The polynomial $(1 - t/c)^k$ is precisely the residual polynomial of Richardson's method with $\omega = 1/c$, so Corollary 3.8 says only that GMRES is at least as good as the best Richardson iteration. The next statement, which needs no eigenvalue information at all, makes that comparison exact.

Theorem 3.9 (GMRES is never worse than a stationary iteration)**.** *Let x_k^R be the Richardson iterates* (1.1) *from $x_0 = 0$ with any ω, and x_k^G the GMRES iterates. Then $\|b - Ax_k^G\|_2 \le \|b - Ax_k^R\|_2$ for every k.*

Proof. $x_k^R \in \mathcal{K}_k(A, b)$ by Theorem 1.5, and x_k^G minimizes the residual over $\mathcal{K}_k(A, b)$. □

Remark 3.10 (Non-normal matrices). For a normal matrix V is unitary and $\kappa_2(V) = 1$, so the eigenvalues determine the bound completely. For highly non-normal A the factor $\kappa_2(V)$ can be enormous and the bound useless; it is known (Greenbaum, Pták and Strakoš 1996) that for any prescribed set of eigenvalues and any non-increasing sequence of residual norms there is a matrix with those eigenvalues on which GMRES produces exactly that sequence. In the non-normal case the eigenvalues alone do not control convergence, and one must look at quantities such as the field of values or pseudospectra of A.

Example 3.11 (Three distinct eigenvalues)**.** Let $A = V\Lambda V^{\mathsf{T}}$ be 50×50 with V a random orthogonal matrix and each diagonal entry of Λ chosen at random from $\{1, 2, 3\}$. For a random b the GMRES residual norms are $8.37,\ 2.51,\ 0.91,\ 6 \times 10^{-15}$: exact after three steps, as the first corollary predicts, although the matrix is 50×50 and dense.

Example 3.12 (Clustered versus spread spectrum)**.** Let $A = V\Lambda V^{\mathsf{T}}$ be 100×100 symmetric with V random orthogonal and 100 eigenvalues equally spaced in an interval $[\lambda_{\min}, \lambda_{\max}]$. Corollary 3.8 with $c = (\lambda_{\min} + \lambda_{\max})/2$ and $\rho = (\lambda_{\max} - \lambda_{\min})/2$ predicts the factor ρ/c per step. For a random unit b:

k	$[1,3]$, factor $\rho/c = 0.5$	$[0.2,3.8]$, factor $\rho/c = 0.9$
1	2.9×10^{-1}	4.7×10^{-1}
2	8.7×10^{-2}	3.1×10^{-1}
5	1.5×10^{-3}	7.5×10^{-2}
10	2.2×10^{-6}	8.9×10^{-3}
20	3.3×10^{-12}	6.0×10^{-5}
40	1.3×10^{-15}	4.6×10^{-10}

Both columns beat the disk bound by a wide margin (for the first column $0.5^{10} \approx 10^{-3}$, while the observed residual is 10^{-6}), because the optimal polynomial on an interval is a scaled Chebyshev polynomial, not $(1 - t/c)^k$. The Chebyshev bound gives the factor $(\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$ with $\kappa = \lambda_{\max}/\lambda_{\min}$, which is 0.27 and 0.63 for the two columns and matches the observed decay much better. This is the same factor that governs conjugate gradients.

Example 3.13 (Eigenvalues can lie)**.** Let $A = I - 2S$ be 20×20, where S has ones on the superdiagonal and zeros elsewhere, and let $b = e_{20}$. Every eigenvalue of A equals 1, so a naive reading of the corollaries would predict convergence in one step. Instead the GMRES residual norms are

$$\|r_1\|_2 = 0.894, \qquad \|r_5\|_2 = \|r_{10}\|_2 = \|r_{19}\|_2 = 0.866, \qquad \|r_{20}\|_2 = 4 \times 10^{-10} :$$

nineteen steps of stagnation, then the exact answer. The matrix is not diagonalizable (it is a single Jordan block), so Theorem 3.6 does not apply, and $\kappa_2(A) \approx 2 \times 10^6$. The grade of b is 20 because $A^{-1} = I + 2S + 4S^2 + \cdots + 2^{19}S^{19}$ and every power of S is needed to express $A^{-1}e_{20}$.

3.7 Cost, memory, restarts, and preconditioning

Step k of GMRES costs one product with A, about $2kn$ flops of orthogonalization, and $O(k)$ flops for the rotations, and it stores $k + 1$ vectors of length n. After many steps the orthogonalization and the storage dominate. The usual remedy is *restarted* GMRES(m): run m steps, form x_m, and start again with $x_0 = x_m$ and b replaced by the new residual. Restarting discards the Krylov history, so it loses the monotone and finite-termination guarantees and can stagnate; m between 20 and 50 is common.

Example 3.14 (The price of restarting)**.** On the matrix with spectrum $[0.2, 3.8]$ from Example 3.12, full GMRES and GMRES(5) compare as follows after k products with A.

k	Full GMRES	GMRES(5)
5	7.5×10^{-2}	7.5×10^{-2}
10	8.9×10^{-3}	1.1×10^{-2}
20	6.0×10^{-5}	3.3×10^{-4}
30	3.6×10^{-7}	1.1×10^{-5}
40	4.6×10^{-10}	3.9×10^{-7}

The first cycle is identical. After that the restarted method, which only ever optimizes over a five-dimensional space, falls behind by a factor that grows with k; it still converges here because the spectrum is well separated from 0, but on harder problems GMRES(m) with small m can stall completely. The restarted method stores 6 vectors instead of 41.

The other remedy is to change the problem. A *preconditioner* is a matrix P such that $P^{-1}v$ is cheap and $P^{-1}A$ has eigenvalues clustered near 1, and GMRES is applied to $P^{-1}Ax = P^{-1}b$. By Theorem 3.6 clustering is exactly what makes GMRES fast. Jacobi ($P = D$), Gauss–Seidel and incomplete factorizations are classical choices; learned preconditioners are a current research topic.

3.8 Symmetric matrices: MINRES and conjugate gradients

When A is symmetric, H_k is tridiagonal, Arnoldi becomes the three-term Lanczos recurrence, and GMRES becomes MINRES, with $O(n)$ storage per step instead of $O(kn)$. When A is in addition positive definite, minimizing the A-norm of the error instead of the 2-norm of the residual gives the conjugate gradient method, whose convergence factor is $(\sqrt{\kappa_2(A)}-1)/(\sqrt{\kappa_2(A)}+1)$. We will not cover these in class; Trefethen and Bau, Lecture 38, is a self-contained account.

4 From linear systems to fixed points: toward deep equilibrium models

4.1 Fixed-point problems

The iterative methods unit started from the idea of writing $Ax = b$ as a fixed-point problem $x = Mx + c$. Deep equilibrium models (DEQs, Bai, Kolter and Koltun 2019) turn this around: a *layer* is defined as the solution of a fixed-point problem. Given an input x and parameters θ, the output is the vector $z^\star$ with

$$z^\star = f(z^\star, x; \theta), \tag{4.1}$$

for example $f(z, x; \theta) = \tanh(Wz + Ux + c)$ with $\theta = (W, U, c)$. Applying f repeatedly, $z_{j+1} = f(z_j, x; \theta)$, is a weight-tied network of infinite depth; a DEQ computes its limit directly by a root finder and never stores the intermediate layers. Two numerical questions arise:

1. *Forward pass.* Find $z^\star$. This is a nonlinear fixed-point problem, solved by fixed-point iteration, Anderson acceleration, or a (quasi-)Newton method.
2. *Backward pass.* Given the gradient $v = \partial \ell / \partial z^\star$ of a loss ℓ with respect to the output, compute the gradient with respect to θ. This, it turns out, requires solving a *linear* system with a matrix that is available only through products.

Everything from the two iterative units applies. If $J = \partial f / \partial z$ satisfies $\|J\| \le L < 1$ in a neighborhood (a contraction), the fixed-point iteration converges linearly with factor L (Banach's theorem). For the *linear* iteration that appears in the backward pass, the convergence theorem of the previous unit gives the sharp condition $\rho(J) < 1$.

Example 4.1 (A scalar layer)**.** Let $f(z; w) = \tanh(wz + u)$ with $w = \frac{1}{2}$, $u = 1$. Fixed-point iteration from $z_0 = 0$ gives

$$0.761594,\ 0.881130,\ 0.893811,\ 0.895079,\ 0.895205,\ 0.895218,\ 0.895219,\ 0.895219,$$

so $z^\star \approx 0.895219$. Here $f'(z^\star) = w(1 - z^{\star 2}) \approx 0.5 \times 0.1986 = 0.0993$, so each step reduces the error by a factor of about 10, which the digits confirm: one new correct digit per step.

Example 4.2 (A two-dimensional layer)**.** Throughout this section we use

$$f(z) = \tanh(Wz + c), \qquad W = \begin{pmatrix} 0.3 & 0.5 \\ -0.4 & 0.2 \end{pmatrix}, \qquad c = \begin{pmatrix} 1 \\ -0.5 \end{pmatrix}.$$

The fixed point is $z^\star \approx (0.6875,\, -0.7260)^\mathsf{T}$, the diagonal matrix of activation derivatives is $D = \operatorname{diag}(1 - z_i^{\star 2}) \approx \operatorname{diag}(0.5273,\, 0.4729)$, and

$$J = DW \approx \begin{pmatrix} 0.1582 & 0.2636 \\ -0.1892 & 0.0946 \end{pmatrix}, \qquad \rho(J) \approx 0.255, \qquad \|J\|_2 \approx 0.308.$$

4.2 Forward solvers

Fixed-point iteration $z_{j+1} = f(z_j)$ costs one evaluation of f per step and converges linearly with factor $\rho(J)$ (when it converges). On Example 4.2 from $z_0 = 0$, the residual norms $\|f(z_j) - z_j\|_2$ are

j	Residual $\|f(z_j) - z_j\|_2$
1	2.5×10^{-1}
2	6.3×10^{-2}
3	1.8×10^{-2}
5	1.0×10^{-3}
10	1.0×10^{-6}
15	1.3×10^{-9}
20	1.3×10^{-12}

The ratio of successive residuals settles at about $0.25 = \rho(J)$, as the theory says.

Newton's method solves $F(z) = z - f(z) = 0$ by

$$z_{j+1} = z_j - (I - J_j)^{-1}\big(z_j - f(z_j)\big), \qquad J_j = \frac{\partial f}{\partial z}(z_j), \tag{4.2}$$

and converges quadratically: on Example 4.2 from $z_0 = 0$ the residuals are 7.2×10^{-2}, 6.1×10^{-4}, 3.2×10^{-8}, 1.1×10^{-16}, the digits doubling each step. The price is a linear solve with $I - J_j$ per step. In a DEQ the dimension n is the width of the layer, J_j is never formed, and the natural way to do this solve is GMRES with the matrix-free product $u \mapsto u - J_j u$: this combination is called *Newton–Krylov*. Quasi-Newton methods (Broyden's method, used in the original DEQ paper) replace $(I - J_j)^{-1}$ by a cheap low-rank approximation that is updated from the iterates and needs no linear solve at all.

Anderson acceleration (Anderson 1965; Walker and Ni 2011) is the method of choice in most DEQ code. It keeps the last $m+1$ iterates $z_{j-m}, \dots, z_j$ and their residuals $r_i = f(z_i) - z_i$, finds the affine combination of the residuals with the smallest norm,

$$\alpha = \arg\min\Big\{ \Big\| \sum_{i=j-m}^{j} \alpha_i r_i \Big\|_2 \;:\; \sum_i \alpha_i = 1 \Big\}, \tag{4.3}$$

which is a small least squares problem just like (3.3), and sets $z_{j+1} = \sum_i \alpha_i f(z_i)$, often with a damping parameter. The logic is the GMRES logic: if f were linear, $f(z) = Jz + v$, then

$\sum \alpha_i f(z_i) = f(\sum \alpha_i z_i)$ when $\sum \alpha_i = 1$, so the new iterate is f applied to the point in the affine hull of the previous iterates with the smallest residual. Walker and Ni proved that for a linear f and unlimited memory, Anderson acceleration is equivalent to GMRES applied to $(I - J)z = v$: the Anderson iterate is f applied to the GMRES iterate. For nonlinear f it behaves like a Krylov method restarted every m steps.

Example 4.3 (Three forward solvers on the same problem)**.** On Example 4.2 from $z_0 = 0$, counting one evaluation of f per step:

Evaluations of f	Fixed-point iteration	Anderson, $m = 2$	Newton
1	2.5×10^{-1}	2.5×10^{-1}	7.2×10^{-2}
2	6.3×10^{-2}	6.4×10^{-2}	6.1×10^{-4}
3	1.8×10^{-2}	6.3×10^{-3}	3.2×10^{-8}
5	1.0×10^{-3}	1.1×10^{-6}	$< 10^{-16}$
8	4×10^{-5}	$< 10^{-13}$	
10	1.0×10^{-6}		

Newton needs the fewest evaluations but each one comes with a Jacobian and a linear solve; Anderson gets the same accuracy in eight evaluations with nothing but a 2×2 least squares problem per step; fixed-point iteration needs about twenty. In a DEQ with n in the thousands, Anderson's cost per step is essentially that of fixed-point iteration, which is why it wins.

Remark 4.4 (When $\rho(J) \geq 1$). Nothing in the definition (4.1) guarantees that a fixed point exists, is unique, or is found by iteration. If W in Example 4.2 is multiplied by 4, fixed-point iteration from 0 never settles but wanders among several points, and Newton's method from a poor start diverges. DEQ architectures therefore include a mechanism that keeps J small: spectral normalization (which bounds $\|W\|_2$ with the power method), Jacobian regularization, or monotone operator parameterizations. This is where the next lectures start.

4.3 What automatic differentiation provides

Reverse-mode automatic differentiation (backpropagation) never forms a Jacobian. For a map $f : \mathbb{R}^n \to \mathbb{R}^n$ it provides two products:

- the Jacobian–vector product $v \mapsto Jv$ (forward mode, `jax.jvp`), and
- the vector–Jacobian product $u \mapsto J^\mathsf{T} u$ (reverse mode, `jax.vjp` or `torch.autograd.grad`),

each at a cost of a small constant times one evaluation of f. For $f(z) = \tanh(Wz + c)$ these are explicit: $J = DW$ with $D = \operatorname{diag}(1 - f_i^2)$, so

$$Jv = D\,(Wv), \qquad J^\mathsf{T} u = W^\mathsf{T}(Du),$$

one matrix–vector product with W or W^T each. The same holds for the parameter Jacobian: the derivative of f in the direction ΔW is $D\,\Delta W\, z$, and its adjoint maps g to $(Dg)z^\mathsf{T}$, an outer product.

This is exactly the *matrix-free* setting of Section 1. A Krylov method sees $I - J^\mathsf{T}$ only through $u \mapsto u - J^\mathsf{T} u$, which autodiff supplies.

4.4 Implicit differentiation

Theorem 4.5 (Implicit function theorem for a fixed point)**.** *Suppose f is continuously differentiable, $z^\star = f(z^\star, x; \theta)$, and $I - J$ is nonsingular, where $J = \partial f/\partial z$ evaluated at $(z^\star, x; \theta)$. Then near θ the solution $z^\star(\theta)$ is a continuously differentiable function of θ with*

$$\frac{\partial z^\star}{\partial \theta} = (I - J)^{-1} \frac{\partial f}{\partial \theta}. \tag{4.4}$$

Derivation of the formula. Differentiate the identity $z^\star(\theta) = f(z^\star(\theta), x; \theta)$ with the chain rule:

$$\frac{\partial z^\star}{\partial \theta} = J \frac{\partial z^\star}{\partial \theta} + \frac{\partial f}{\partial \theta}, \qquad \text{so} \qquad (I - J)\frac{\partial z^\star}{\partial \theta} = \frac{\partial f}{\partial \theta}.$$

(The existence and differentiability of $z^\star(\theta)$ is the standard implicit function theorem applied to $F(z, \theta) = z - f(z, x; \theta)$, whose z-derivative $I - J$ is nonsingular.) □

Example 4.6 (The scalar layer, differentiated)**.** In Example 4.1, $\partial f/\partial w = z(1 - \tanh^2(wz + u)) = z(1 - z^2)$ at the fixed point and $J = w(1 - z^{\star 2})$, so

$$\frac{dz^\star}{dw} = \frac{z^\star(1 - z^{\star 2})}{1 - w(1 - z^{\star 2})} \approx \frac{0.895219 \times 0.198582}{1 - 0.5 \times 0.198582} \approx 0.197372.$$

A central finite difference of the converged fixed point, $(z^\star(w + h) - z^\star(w - h))/2h$ with $h = 10^{-6}$, gives 0.197372 as well. Note what the formula does *not* contain: the number of iterations used to find $z^\star$, or any of the intermediate iterates.

For a scalar loss $\ell(z^\star)$ with $v = \nabla_z \ell \in \mathbb{R}^n$, the chain rule and (4.4) give

$$\nabla_\theta \ell = \left(\frac{\partial f}{\partial \theta}\right)^{\mathsf{T}} (I - J)^{-\mathsf{T}} v = \left(\frac{\partial f}{\partial \theta}\right)^{\mathsf{T}} g, \qquad \text{where} \qquad (I - J^{\mathsf{T}})\, g = v. \tag{4.5}$$

Once g is known, the product $(\partial f/\partial \theta)^{\mathsf{T}} g$ is a single vector–Jacobian product. The whole difficulty of training a DEQ is therefore the linear system

$$(I - J^{\mathsf{T}})\, g = v, \tag{4.6}$$

and this system is *matrix-free*: J has n^2 entries that nobody wants to form, but $J^{\mathsf{T}} u$ for any u is one vector–Jacobian product. The memory cost of the backward pass is independent of how many fixed-point iterations the forward pass took, which is the main practical advantage of a DEQ over an unrolled deep network.

Example 4.7 (The two-dimensional layer, differentiated)**.** Take the loss $\ell = \frac{1}{2}\|z^\star\|_2^2$ in Example 4.2, so $v = z^\star$. Solving $(I - W^{\mathsf{T}} D)g = z^\star$ and forming $\nabla_W \ell = (Dg)z^{\star\mathsf{T}}$ gives

$$\nabla_W \ell \approx \begin{pmatrix} 0.3392 & -0.3582 \\ -0.1721 & 0.1818 \end{pmatrix},$$

and central finite differences of the loss (with 500 fixed-point iterations per evaluation) agree to all four digits shown.

Example 4.8 (Unrolled versus implicit gradients)**.** The alternative to implicit differentiation is to treat the K fixed-point iterations as K layers and backpropagate through all of them, which stores K intermediate vectors. For Example 4.7:

Unrolled steps K	$\|z_K - z^\star\|_2$	$\|\nabla_W^{\text{unrolled}} - \nabla_W^{\text{implicit}}\|_F$
1	2.7×10^{-1}	5.5×10^{-1}
2	7.4×10^{-2}	2.5×10^{-1}
5	1.2×10^{-3}	9.9×10^{-3}
10	1.1×10^{-6}	1.5×10^{-5}
20	1.5×10^{-12}	5.1×10^{-11}
30	0	1.8×10^{-16}

The unrolled gradient converges to the implicit one as $K \to \infty$, at the same rate $\rho(J)$ as the iterates, but it costs $O(Kn)$ memory and the gradient of the *truncated* network is not the gradient of the *equilibrium*. Implicit differentiation gets the exact limit for the price of one linear solve, with $O(n)$ memory. Backpropagating through the unrolled iteration is, incidentally, exactly the Neumann series of the next subsection: the adjoint recursion $a_j = J_{j+1}^{\mathsf{T}} a_{j+1}$ accumulates $v + J^{\mathsf{T}}v + (J^{\mathsf{T}})^2 v + \dots$ when all the J_j are equal.

4.5 Neumann series versus GMRES for the backward solve

The cheapest way to solve (4.6) is the fixed-point iteration

$$g_{j+1} = J^{\mathsf{T}} g_j + v, \qquad g_0 = 0, \qquad \text{so} \qquad g_k = \sum_{j=0}^{k-1} (J^{\mathsf{T}})^j v, \tag{4.7}$$

which is Richardson's method with $\omega = 1$ for $A = I - J^{\mathsf{T}}$, that is, the Neumann series (1.2). By the convergence theorem for stationary iterations it converges for every v if and only if $\rho(J) < 1$, with asymptotic factor $\rho(J)$. Truncating it after one term, $g \approx v$, is the *one-step gradient* used in some recent recursive reasoning models; it is cheap but it is only accurate when J is small. (In Example 4.7, $g \approx v$ gives a gradient with a relative error of roughly $\rho(J) \approx 25\%$.)

Theorem 3.9 applies directly: the Neumann iterate g_k lies in $\mathcal{K}_k(I - J^{\mathsf{T}}, v)$, so GMRES with the same k products $J^{\mathsf{T}}u$ has a residual no larger than the Neumann series, and Corollary 3.8 with $c = 1$, $\rho = \rho(J)$ reproduces the factor $\rho(J)^k$ as a worst case. In practice GMRES is much better than the worst case, because it adapts to the actual spectrum of J instead of using the fixed polynomial $(1 - t)^k$.

Example 4.9. Let $n = 200$ and let J be a random matrix (entries drawn from the standard normal distribution with seed 0, divided by $\sqrt{n}$) scaled so that $\rho(J) = 0.9$. Then $\|J\|_2 \approx 1.67 > 1$, so J is *not* a contraction in the 2-norm, yet the Neumann series converges because $\rho(J) < 1$. Solving $(I - J^{\mathsf{T}})g = v$ for a random unit vector v gives the residual norms below.

Products with J^{T} (k)	Neumann series $\|v - (I - J^{\mathsf{T}})g_k\|_2$	GMRES $\|r_k\|_2$
1	8.8×10^{-1}	6.4×10^{-1}
2	7.4×10^{-1}	4.4×10^{-1}
5	5.1×10^{-1}	2.1×10^{-1}
10	2.1×10^{-1}	1.1×10^{-1}
20	4.3×10^{-2}	2.3×10^{-2}
30	1.2×10^{-2}	1.8×10^{-3}
40	3.7×10^{-3}	1.5×10^{-4}

Both methods cost one vector–Jacobian product per step. After 40 products GMRES is about 25 times more accurate, and the gap keeps widening.

4.6 Matrix-free GMRES in SciPy

The backward solve (4.6) needs only a function that computes $u \mapsto J^{\mathsf{T}}u$. In PyTorch this is `torch.autograd.grad(f, z, u)`; in JAX it is `jax.vjp`. Wrapped as a `LinearOperator`, it can be handed to SciPy's GMRES. The sketch below uses NumPy and an explicit Jacobian only so that it runs on its own; it is the 200-dimensional version of Example 4.7.

```
import numpy as np
from scipy.sparse.linalg import LinearOperator, gmres

def deq_backward(vjp, v, rtol=1e-8, restart=20):
    """Solve (I - J^T) g = v given only vjp(u) = J^T u.

    vjp is a function returning the vector-Jacobian product, which an
    autodiff library provides without forming J.  Returns g and the
    SciPy exit flag (0 means converged).
    """
    n = v.size
    M = LinearOperator((n, n), matvec=lambda u: u - vjp(u),
                       dtype=v.dtype)
    g, info = gmres(M, v, rtol=rtol, restart=restart, maxiter=200)
    return g, info

# Example with f(z) = tanh(W z + c): J = diag(1 - z*^2) W at the fixed point.
rng = np.random.default_rng(0)
n = 200
W = rng.standard_normal((n, n)) / np.sqrt(n) * 0.5   # ||W||_2 < 1
c = rng.standard_normal(n)
z = np.zeros(n)
for _ in range(100):                     # forward pass: fixed-point iteration
    z = np.tanh(W @ z + c)
D = 1.0 - z**2
vjp = lambda u: W.T @ (D * u)            # J^T u = W^T diag(D) u
v = rng.standard_normal(n)               # dloss/dz*
g, info = deq_backward(vjp, v)
```

```
grad_W = np.outer(D * g, z)              # dloss/dW = (D g) z*^T
print(info, np.linalg.norm(v - (g - vjp(g))))
```

The last line prints the exit flag and the residual of the linear solve. The gradient with respect to W is $(Dg)z^{\star\mathsf{T}}$ because the directional derivative of f in the direction ΔW is $D\,\Delta W\,z^\star$, and the adjoint of $\Delta W \mapsto D\,\Delta W\,z^\star$ is $g \mapsto (Dg)z^{\star\mathsf{T}}$.

The forward solvers of this section are equally short. The following Anderson acceleration routine produced the table above; `m` is the memory, and the least squares problem (4.3) is solved in its unconstrained form by eliminating the constraint $\sum_i \alpha_i = 1$ with differences of consecutive residuals.

```
def anderson(f, z0, m=5, tol=1e-10, maxit=100):
    """Anderson acceleration for z = f(z); returns z and residual norms."""
    zs, fs = [z0], [f(z0)]
    hist = [np.linalg.norm(fs[0] - zs[0])]
    for k in range(maxit):
        mk = min(m, k)
        if mk == 0:
            z_new = fs[-1]
        else:
            R = np.array([fs[j] - zs[j] for j in range(-mk - 1, 0)]).T
            F = np.array(fs[-mk - 1:]).T
            dR, dF = R[:, 1:] - R[:, :-1], F[:, 1:] - F[:, :-1]
            gamma, *_ = np.linalg.lstsq(dR, R[:, -1], rcond=None)
            z_new = fs[-1] - dF @ gamma
        zs.append(z_new); fs.append(f(z_new))
        hist.append(np.linalg.norm(fs[-1] - zs[-1]))
        if hist[-1] < tol:
            break
    return zs[-1], hist
```

4.7 Checklist: what the deep equilibrium lectures assume

You are ready for the DEQ lectures if you can do the following, all of which are covered in these notes and the two iterative units.

1. State when the fixed-point iteration $z \leftarrow f(z)$ converges (contraction; for linear maps, $\rho(J) < 1$) and at what rate.
2. Run Newton's method and Anderson acceleration on a small fixed-point problem, and explain why Anderson is GMRES in disguise.
3. Explain what a vector–Jacobian product is, why it costs about one evaluation of f, and write it down explicitly for $f(z) = \tanh(Wz + c)$.
4. Derive the implicit gradient formula (4.5) from the identity $z^\star = f(z^\star;\theta)$, and identify the linear system (4.6) that the backward pass must solve.

5. Solve that system by the Neumann series or by GMRES, state which one is never worse and why (Theorem 3.9), and give the convergence rate of each in terms of $\rho(J)$.

6. Explain the difference between the gradient of the unrolled iteration and the implicit gradient, and why the latter needs $O(n)$ memory.

7. Estimate $\|W\|_2$ with the power method, since bounding it is how spectral normalization keeps $\rho(J) < 1$.

The DEQ lectures themselves then cover the model and its training (the original DEQ, multiscale DEQs, monotone operator DEQs), how to guarantee the existence and uniqueness of $z^\star$, Jacobian regularization, inexact gradients when the fixed point is not reached, and the recent recurrent-depth models that reuse these ideas.

5 Practice problems

Problem 1. Show that $\mathcal{K}_k(A, b) = \mathcal{K}_k(A - \alpha I, b)$ for every scalar α. Conclude that GMRES applied to $(A - \alpha I)x = b$ searches the same subspaces as GMRES applied to $Ax = b$.

Problem 2. Let $A = \begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}$ and $b = e_1$. Carry out the Arnoldi iteration by hand and compute the GMRES residual norm after one step. Then show that no Richardson iteration $x_{j+1} = x_j + \omega(b - Ax_j)$ converges for this A, whatever ω, while GMRES solves the system exactly in two steps.

Problem 3. Prove directly from the definition (3.1) that the GMRES residual norms are non-increasing, and that $r_m = 0$ where m is the grade of b with respect to A.

Problem 4. Suppose $A = I - J$ where J is diagonalizable, $J = V\Lambda V^{-1}$, with $\rho(J) = \frac{1}{2}$ and $\kappa_2(V) = 10$. How many GMRES steps guarantee $\|r_k\|_2 \le 10^{-8}\|b\|_2$ by Corollary 3.8? How many terms of the Neumann series guarantee the same by the bound $\|J^k\|_2 \le \kappa_2(V)\rho(J)^k$? Comment on the comparison.

Problem 5. For the matrix $A = \operatorname{diag}(1, 2, 2, 3, 3, 3)$ and $b = (1, 1, 1, 1, 1, 1)^\mathsf{T}$, find the grade of b and the minimal polynomial of b with respect to A without computing any Krylov vectors. After how many steps does GMRES solve $Ax = b$? What if $b = (1, 0, 0, 1, 0, 0)^\mathsf{T}$?

Problem 6. Let $f(z) = \frac{1}{2}\cos z$. Starting from $z_0 = 0$, compute two steps of fixed-point iteration and one step of Newton's method for $z = f(z)$. Then, with $z^\star$ the fixed point, write $dz^\star/d\theta$ for the family $f_\theta(z) = \theta\cos z$ at $\theta = \frac{1}{2}$ in terms of $z^\star$.

Problem 7. Let $z^\star = \tanh(Wz^\star + Ux)$ with $W \in \mathbb{R}^{n\times n}$, and let $D = \operatorname{diag}(1 - (z_i^\star)^2)$. Write down $J = \partial f/\partial z$ at $z^\star$, the linear system solved in the backward pass, and the gradient of a loss with respect to W in terms of its solution g. Show that $\|W\|_2 < 1$ guarantees $\rho(J) < 1$, and name the algorithm from this course that estimates $\|W\|_2$.

Solutions

Problem 1. By Lemma 1.2, $\mathcal{K}_k(A,b)$ is the set of $q(A)b$ with $\deg q \le k-1$. Since $(A-\alpha I)^j$ is a polynomial in A of degree j and conversely $A^j = ((A-\alpha I)+\alpha I)^j$ is a polynomial in $A-\alpha I$ of degree j, the sets of polynomials of degree at most $k-1$ in A and in $A-\alpha I$ coincide. GMRES minimizes the residual over the same subspaces in both cases (the residuals differ, since the right-hand side of the minimization involves $A-\alpha I$).

Problem 2. $q_1 = e_1$, $Aq_1 = (1,2)^{\mathsf T}$, $h_{11} = 1$, $v = (0,2)^{\mathsf T}$, $h_{21} = 2$, $q_2 = e_2$; then $Aq_2 = (2,1)^{\mathsf T}$, $h_{12} = 2$, $h_{22} = 1$, $v = 0$, breakdown at $k = 2$ with $H_2 = A$. GMRES step 1 minimizes $\|(1,0)^{\mathsf T} - y(1,2)^{\mathsf T}\|_2$: $5y = 1$, $y = 1/5$, residual norm $\sqrt{(4/5)^2 + (2/5)^2} = 2/\sqrt{5} \approx 0.894$. The eigenvalues of A are 3 and -1, so $I - \omega A$ has eigenvalues $1 - 3\omega$ and $1 + \omega$; $|1 - 3\omega| < 1$ needs $0 < \omega < 2/3$ while $|1 + \omega| < 1$ needs $-2 < \omega < 0$, which is impossible. GMRES terminates at the breakdown, $k = 2$, with $y = H_2^{-1} e_1 = (-1/3, 2/3)^{\mathsf T}$ and $x_2 = (-1/3, 2/3)^{\mathsf T}$.

Problem 3. $\mathcal{K}_k \subseteq \mathcal{K}_{k+1}$, so the minimum of $\|b - Ax\|_2$ over $\mathcal{K}_{k+1}$ is at most the minimum over $\mathcal{K}_k$. By Theorem 1.3, $A^{-1}b \in \mathcal{K}_m$, and choosing $x = A^{-1}b$ gives residual zero, so the minimum over $\mathcal{K}_m$ is zero.

Problem 4. The eigenvalues of A are $1 - \mu_i$ with $|\mu_i| \le \tfrac{1}{2}$, so they lie in the disk of center 1 and radius $\tfrac{1}{2}$, and Corollary 3.8 gives $\|r_k\|_2/\|b\|_2 \le 10 \cdot 2^{-k}$; this is at most 10^{-8} when $2^k \ge 10^9$, that is $k \ge 30$. The Neumann residual is $v - (I - J^{\mathsf T})g_k = (J^{\mathsf T})^k v$, and $\|J^k\|_2 \le \kappa_2(V)\rho(J)^k = 10 \cdot 2^{-k}$ gives the same $k \ge 30$. The two *bounds* coincide because the bound for GMRES was obtained with the Richardson polynomial $(1-t)^k$; Theorem 3.9 says the actual GMRES residual is never larger than the Neumann one, and usually it is far smaller.

Problem 5. The distinct eigenvalues are $1, 2, 3$ and b has a nonzero component in each eigenspace, so the minimal polynomial is $(t-1)(t-2)(t-3)$, the grade is 3, and GMRES terminates after three steps. For $b = (1,0,0,1,0,0)^{\mathsf T}$ only the eigenspaces of 1 and 3 are present, the minimal polynomial is $(t-1)(t-3)$, and GMRES terminates after two steps.

Problem 6. Fixed-point iteration: $z_1 = \tfrac{1}{2}\cos 0 = \tfrac{1}{2}$, $z_2 = \tfrac{1}{2}\cos\tfrac{1}{2} \approx 0.43879$. Newton: $F(z) = z - \tfrac{1}{2}\cos z$, $F'(z) = 1 + \tfrac{1}{2}\sin z$, so $z_1 = 0 - F(0)/F'(0) = \tfrac{1}{2}$ (the same first step, since $F'(0) = 1$), and $z_2 = \tfrac{1}{2} - F(\tfrac{1}{2})/F'(\tfrac{1}{2}) = \tfrac{1}{2} - 0.06121/1.23971 \approx 0.45063$; the fixed point is $z^\star \approx 0.45018$. Implicit differentiation of $z^\star = \theta\cos z^\star$: $dz^\star/d\theta = \cos z^\star - \theta \sin z^\star \, dz^\star/d\theta$, so $dz^\star/d\theta = \cos z^\star/(1 + \theta \sin z^\star) \approx 0.90038/1.21761 \approx 0.7395$.

Problem 7. $J = DW$. The backward pass solves $(I - W^{\mathsf T}D)g = v$ with $v = \nabla_z \ell$. Since the derivative of f in the direction ΔW is $D\,\Delta W\, z^\star$, the gradient is $\nabla_W \ell = (Dg)\, z^{\star\mathsf T}$. Every entry of D lies in $(0,1]$, so $\|D\|_2 \le 1$ and $\rho(J) \le \|DW\|_2 \le \|D\|_2\|W\|_2 < 1$. The power method applied to $W^{\mathsf T}W$ (equivalently, to the pair of products $u \mapsto W^{\mathsf T}(Wu)$) estimates $\|W\|_2 = \sigma_1(W)$; this is what spectral normalization does.

References

D. G. Anderson, Iterative procedures for nonlinear integral equations, *Journal of the ACM* 12 (1965), 547–560.

S. Bai, J. Z. Kolter and V. Koltun, Deep equilibrium models, *Advances in Neural Information Processing Systems* 32 (2019).

A. Greenbaum, V. Pták and Z. Strakoš, Any nonincreasing convergence curve is possible for GMRES, *SIAM Journal on Matrix Analysis and Applications* 17 (1996), 465–469.

Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed., SIAM, 2003, Chapters 6 and 9.

Y. Saad and M. H. Schultz, GMRES: A generalized minimal residual algorithm for solving nonsymmetric linear systems, *SIAM Journal on Scientific and Statistical Computing* 7 (1986), 856–869.

L. N. Trefethen and D. Bau, III, *Numerical Linear Algebra*, SIAM, 1997, Lectures 33–35 and 38.

H. F. Walker and P. Ni, Anderson acceleration for fixed-point iterations, *SIAM Journal on Numerical Analysis* 49 (2011), 1715–1735.